Universidade Federal de Ouro Preto - UFOP
Instituto de Ciências Exatas e Biológicas - ICEB
Departamento de Computação - DECOM
Ciência da Computação

Trabalho Prático I (TP I)- Pesquisa Externa

BCC203 - Estrutura de Dados II

Pedro Augusto Santchelis Cândido, Miguel Souza Cocato, Kevilin Pereira Brites, Pablo Henrique Euzebio, Guilherme Figueiredo

Professor: Guilherme Tavares de Assis

Ouro Preto
15 de outubro de 2026

Sumário

1 Introdução **1**

2 Desenvolvimento **1**
2.1 Estruturas básicas . . . 1
2.2 Geração dos arquivos . . . 2
2.3 Programa principal . . . 2
2.4 Método 1 – Acesso sequencial indexado . . . 3
2.5 Método 2 – Árvore binária de pesquisa em memória externa . . . 4
2.6 Método 3 – Árvore B . . . 5
2.7 Método 4 – Árvore B* . . . 6
2.8 Contagem dos quesitos . . . 7

3 Resultado de cada método individualmente: **7**
3.1 Acesso sequencial indexado: . . . 8
3.1.1 Arquivo ordenado ascendentemente: . . . 8
3.1.2 Arquivos ordenados descendentemente e aleatoriamente: . . . 8
3.1.3 Análise (Acesso sequencial indexado): . . . 8
3.2 Árvore binária de pesquisa adequada a memória externa: . . . 9
3.2.1 Arquivo ordenado ascendentemente: . . . 9
3.2.2 Arquivo ordenado descendentemente: . . . 9
3.2.3 Arquivo ordenado aleatoriamente: . . . 10
3.2.4 Análise (Árvore binária): . . . 10
3.3 Árvore B: . . . 11
3.3.1 Arquivo ordenado ascendentemente: . . . 11
3.3.2 Arquivo ordenado descendentemente: . . . 11
3.3.3 Arquivo ordenado aleatoriamente: . . . 12
3.3.4 Análise (Árvore B): . . . 12
3.4 Árvore B*: . . . 13
3.4.1 Arquivo ordenado ascendentemente: . . . 13
3.4.2 Arquivo ordenado descendentemente: . . . 13
3.4.3 Arquivo ordenado aleatoriamente: . . . 14
3.4.4 Análise (Árvore B*): . . . 14

4 Análise comparativa entre os métodos **15**

5 Conclusão **16**
5.1 Impressões Gerais: . . . 16

Lista de Tabelas

1 Organização dos arquivos do projeto . . . 1
2 O que é contado em cada método . . . 7
3 Acesso sequencial indexado – arquivo ordenado ascendentemente (média de 20 chaves) . . . 8
4 Árvore binária externa – arquivo ordenado ascendentemente (média de 20 chaves) . . . 9
5 Árvore binária externa – arquivo ordenado descendentemente (média de 20 chaves) . . . 9
6 Árvore binária externa – arquivo ordenado aleatoriamente (média de 20 chaves) . . . 10
7 Árvore B ($m = 2$) – arquivo ordenado ascendentemente (média de 20 chaves) . . . 11
8 Árvore B ($m = 2$) – arquivo ordenado descendentemente (média de 20 chaves) . . . 11
9 Árvore B ($m = 2$) – arquivo ordenado aleatoriamente (média de 20 chaves) . . . 12
10 Árvore B* – arquivo ordenado ascendentemente (média de 20 chaves) . . . 13
11 Árvore B* – arquivo ordenado descendentemente (média de 20 chaves) . . . 13
12 Árvore B* – arquivo ordenado aleatoriamente (média de 20 chaves) . . . 14
13 Comparação dos quatro métodos com 200 000 registros (média de 20 chaves) . . . 15

Lista de Códigos Fonte

1 Estruturas compartilhadas (`base.h`) . . . 1
2 Criação e embaralhamento das chaves (`geradorTradutor.c`) . . . 2
3 Seleção do método (`main.c`), resumido . . . 3
4 Construção da tabela de índices (`acesso_sequencial.h`) . . . 3
5 Localização da página e leitura (`acesso_sequencial.h`) . . . 3
6 Nó da árvore binária em arquivo (`arvoreBinaria.h`) . . . 4
7 Atualização do pai após a inserção (`arvoreBinaria.h`), resumido . . . 4
8 Página da árvore B (`arvoreB.h`) . . . 5
9 Divisão de página cheia (`arvoreB.h`), resumido . . . 5
10 Página da árvore B* (`arvoreBestrelaplus.h`) . . . 6
11 Pesquisa na árvore B* (`arvoreBestrelaplus.h`), resumido . . . 6
12 Divisão de folha na árvore B* (`arvoreBestrelaplus.h`), resumido . . . 7

1 Introdução

O trabalho consiste em duas partes, a implementação de quatro métodos de pesquisa: (1) acesso sequencial indexado, (2) árvore binária de pesquisa adequada à memória externa, (3) árvore B e (4) árvore B*, e o estudo da complexidade e análise experimental da complexidade de desempenho de cada um deles individualmente.

Os métodos foram implementados em linguagem C e operam sobre arquivos binários de registros armazenados em memória externa. Cada registro é composto por uma chave inteira chave(int 32bits), um dado inteiro longo dado1(long int value 64bits) e uma cadeia de 5000 caracteres dado2(array de char 5000 bytes), totalizando aproximadamente 5.012 bytes por registro.

A análise experimental avalia, separadamente, a etapa de criação dos índices (quando aplicável) e a etapa de pesquisa. Os quesitos considerados foram o número de transferências (leituras) da memória externa para a interna, o número de comparações entre chaves de pesquisa e o tempo de execução. Os testes foram realizados com arquivos de 100, 200, 2.000, 20.000, 200.000 e 2.000.000 de registros, em diferentes situações de ordem: ascendente, descendente e aleatória, quando aplicáveis a cada método. Para cada combinação de parâmetros, foram pesquisadas automaticamente 20 chaves distintas e existentes no arquivo, e os resultados correspondem à média de cada quesito. O programa é executado pela linha de comando pesquisa <método> <quantidade> <situação> <chave> [-P], que informa se a chave foi encontrada e retorna os valores dos quesitos analisados.

2 Desenvolvimento

O código foi organizado de forma modular: cada método de pesquisa está em um arquivo de cabeçalho próprio, e todos compartilham as definições de `base.h`. A geração dos arquivos de teste é feita por um programa separado (`geradorTradutor.c`).

Tabela 1: Organização dos arquivos do projeto

Arquivo	**Responsabilidade**
`base.h`	Estrutura do registro, estrutura de análise (contadores e tempo)
`geradorTradutor.c`	Geração dos arquivos binários (crescente, decrescente, aleatório) e conversão para texto
`main.c`	Leitura dos argumentos da linha de comando e chamada do método escolhido
`acesso_sequencial.h`	Método 1 – acesso sequencial indexado
`arvoreBinaria.h`	Método 2 – árvore binária de pesquisa em memória externa
`arvoreB.h`	Método 3 – árvore B (páginas gravadas em arquivo)
`arvoreBestrelaplus.h`	Método 4 – árvore B*

2.1 Estruturas básicas

O registro segue o formato pedido no enunciado. A estrutura `Analise` acumula os três quesitos medidos: transferências, comparações e tempo. O tempo é medido com `clock_gettime(CLOCK_MONOTONIC)`, que não sofre ajustes do relógio do sistema; no Windows (MinGW), `base.h` substitui essa função pelo contador de alta precisão `QueryPerformanceCounter`.

```
1 #define TAM_DADO2 5000
2
3 typedef struct registros{
```

```
    int chave;
    long int dado1;
    char dado2[TAM_DADO2];
} Registros;

typedef struct analise{
    long int transferencia;
    long int comparacoes;
    struct timespec inicio;
    struct timespec fim;
    double tempoTotal;
} Analise;
```

Código 1: Estruturas compartilhadas (`base.h`)

Em todos os métodos, a análise é zerada (`inicializaAnalise`) entre a etapa de pré-processamento e a de pesquisa, para que os quesitos de cada etapa sejam medidos separadamente.

2.2 Geração dos arquivos

O gerador cria primeiro um vetor com as chaves de 1 a n, em ordem crescente ou decrescente. Para a situação aleatória, o vetor crescente é embaralhado pelo algoritmo de Fisher–Yates, que garante chaves distintas e custa $O(n)$. Em seguida, cada chave é gravada em um registro completo no arquivo binário.

```
for(int i = 0 ; i < quantidade ; i++){
    if(situacao == 2) chaves[i] = quantidade - i;  // decrescente
    else              chaves[i] = i + 1;           // crescente
}
if(situacao == 3){                                 // aleatorio: Fisher-Yates
    for(int i = quantidade-1; i > 0; i--){
        int j = rand() % (i+1);
        int aux = chaves[i]; chaves[i] = chaves[j]; chaves[j] = aux;
    }
}
for(int i = 0 ; i < quantidade ; i++){
    registro.chave = chaves[i];
    registro.dado1 = rand() % 10000;
    fwrite(&registro, sizeof(Registros), 1, arq);
}
```

Código 2: Criação e embaralhamento das chaves (`geradorTradutor.c`)

2.3 Programa principal

O `main` converte os argumentos com `strtol`, verificando se cada um é um número válido, escolhe o arquivo de acordo com a situação e chama o método correspondente. O argumento opcional `-P` ativa a impressão das chaves e das comparações realizadas durante a pesquisa.

```c
// pesquisa <metodo> <quantidade> <situacao> <chave> [-P]
int metodo     = strtol(argv[1], &check, 10);
int quantidade = strtol(argv[2], &check, 10);
int chave      = strtol(argv[4], &check, 10);
FILE *arq = fopen(situacao, "rb");

if      (metodo == 1) realizaPesquisa(&a, &itemPesquisado, quantidade, opt, arq);
else if (metodo == 2) realizarPesquisaBinaria(&a, &itemPesquisado, quantidade, opt, arq);
else if (metodo == 3) realizaPesquisaArvoreB(&a, &itemPesquisado, quantidade, opt, arq);
else if (metodo == 4) RealizaPesquisaArvoreBEstrela(&a, &itemPesquisado, quantidade, arq, opt);
```

Código 3: Seleção do método (`main.c`), resumido

2.4 Método 1 – Acesso sequencial indexado

Pré-processamento. O arquivo é lido página a página, com 100 registros por página (`ITENSPAGINA`). Para cada página, a primeira chave e o número da página são guardados na tabela de índices, mantida em memória principal. Cada página lida conta como uma transferência, totalizando $\lceil n/100 \rceil$ transferências.

```c
while(cont < tam){
    int restantes = qtdRegistrosArq - cont * ITENSPAGINA;
    int aLer = restantes < ITENSPAGINA ? restantes : ITENSPAGINA;
    if (fread(itens, sizeof(Registros), aLer, arq) == 0) break;
    a->transferencia++;
    tabelaPaginas[cont].chavePag = itens[0].chave;
    tabelaPaginas[cont].pos = cont+1;
    cont++;
}
```

Código 4: Construção da tabela de índices (`acesso_sequencial.h`)

Pesquisa. A tabela é percorrida até encontrar a última página cuja primeira chave é menor ou igual à chave procurada. O deslocamento dessa página no arquivo é calculado em bytes (com `fseek` de 64 bits, pois os arquivos maiores passam de 2 GB), a página é lida com uma única transferência e a chave é procurada sequencialmente dentro dela.

```c
while(IndiceTabelaPagina < tamTabela &&
      tabelaPaginas[IndiceTabelaPagina].chavePag <= itemPesquisado->chave){
    a->comparacoes++;
    IndiceTabelaPagina++;
}
desloc = (long long)(tabelaPaginas[IndiceTabelaPagina-1].pos - 1)
         * sizeof(Registros) * ITENSPAGINA;
SEEK64(arq, desloc, SEEK_SET);
fread(pagina, sizeof(Registros), qtdItens, arq); a->transferencia++;

```

```c
for (int i = 0; i < qtdItens; i++){
    a->comparacoes++;
    if(pagina[i].chave == itemPesquisado->chave){ /* encontrado */ }
}
```

Código 5: Localização da página e leitura (`acesso_sequencial.h`)

2.5 Método 2 – Árvore binária de pesquisa em memória externa

Cada nó da árvore é um `Item` gravado no arquivo `arvoreBinaria.bin`. Em vez de ponteiros de memória, os campos `esq` e `dir` guardam a *posição* do filho no arquivo, e o valor -1 indica ausência de filho.

```c
typedef struct{
    Registros registro;
    int esq;   // posicao do filho esquerdo no arquivo (-1 = nenhum)
    int dir;   // posicao do filho direito no arquivo (-1 = nenhum)
} Item;
```

Código 6: Nó da árvore binária em arquivo (`arvoreBinaria.h`)

Inserção. O novo item é gravado no final do arquivo. Depois, a árvore é percorrida a partir da raiz (posição 0) até achar o nó que receberá o novo filho; esse nó tem o campo `esq` ou `dir` atualizado e é regravado na mesma posição. Cada leitura e cada escrita contam como uma transferência.

```c
fwrite(&novoItem, sizeof(Item), 1, arvore);     // novo item no fim do arquivo
fseek(arvore, 0, SEEK_SET);                     // volta para a raiz
while(1){
    posicaoParaAttArvore = ftell(arvore)/sizeof(Item);
    fread(&ItemAtual, sizeof(Item), 1, arvore);
    if(novoItem.registro.chave < ItemAtual.registro.chave){
        if(ItemAtual.esq == -1){                 // achou o pai
            ItemAtual.esq = posicaoNovoItem;
            fseek(arvore, posicaoParaAttArvore*sizeof(Item), SEEK_SET);
            fwrite(&ItemAtual, sizeof(Item), 1, arvore);
            break;
        }
        fseek(arvore, ItemAtual.esq*sizeof(Item), SEEK_SET);
    }
    else { /* simetrico para a direita */ }
}
```

Código 7: Atualização do pai após a inserção (`arvoreBinaria.h`), resumido

Pesquisa. Segue o mesmo caminho da inserção: a cada nó lido, compara a chave e salta (`fseek`) para a posição do filho esquerdo ou direito. Por isso o custo depende diretamente da altura da árvore: $O(\log n)$ quando os dados chegam em ordem aleatória e $O(n)$ quando chegam ordenados, caso em que a árvore degenera em uma lista.

2.6 Método 3 – Árvore B

A árvore B tem ordem $m = 2$, ou seja, cada página guarda até $2m = 4$ chaves e $2m + 1$ filhos. As páginas ficam gravadas no arquivo `arvoreB.bin`; o "ponteiro" para um filho é o número da página nesse arquivo (`TipoApontador` é um `long`). Para não duplicar os 5 000 bytes de `dado2`, a árvore guarda apenas a chave e a posição do registro no arquivo de dados.

```
typedef struct TipoRegistro{
    int chave;
    long pos;          // indice do registro no arquivo de dados
} TipoRegistro;

typedef long TipoApontador;   // numero da pagina em arvoreB.bin
typedef struct TipoPagina{
    int n;
    TipoRegistro r[MAX_CHAVES];
    TipoApontador p[MAX_CHAVES + 1];
} TipoPagina;
```

Código 8: Página da árvore B (`arvoreB.h`)

As funções `lePaginaB` e `escrevePaginaB` posicionam o arquivo em `pos * sizeof(TipoPagina)` e leem ou gravam uma página inteira. Cada leitura de página conta como uma transferência.

Inserção. A função recursiva `ins` desce até a folha. Na volta da recursão, se a página tem espaço, o item é inserido em ordem (`insereNaPagina`); se a página está cheia, ela é dividida: o item do meio sobe para o pai e os m itens da direita vão para uma nova página, gravada no fim do arquivo. Quando a própria raiz se divide, `insereB` cria uma nova raiz, e a árvore cresce em altura.

```
if (pag.n < MAX_CHAVES){                  // ha espaco: insere e grava
    insereNaPagina(&pag, *regRet, *apRet, a);
    escrevePaginaB(t, ap, &pag);
    *cresceu = false;
    return true;
}
// overflow: divide a pagina
TipoPagina temp; inicializaPaginaB(&temp);
/* ... distribui os itens entre pag e temp ... */
pag.n = M;
temp.p[0] = pag.p[M+1];
*regRet = pag.r[M];                       // item do meio sobe
TipoApontador novo = t->numPaginas++;
*apRet = novo;
escrevePaginaB(t, ap,   &pag);
escrevePaginaB(t, novo, &temp);
*cresceu = true;
```

Código 9: Divisão de página cheia (`arvoreB.h`), resumido

Pesquisa. Em cada página lida, as chaves são comparadas até encontrar a chave procurada ou o ramo por onde descer. Ao encontrar, o campo `pos` permite ler o registro completo no arquivo de dados com uma transferência adicional. O número de páginas lidas é limitado pela altura da árvore, $O(\log_{m+1} n)$, qualquer que seja a ordem dos dados de entrada.

2.7 Método 4 – Árvore B*

Na árvore B* implementada, as páginas são de dois tipos, distinguidos pelo campo `Pt`: as *internas* guardam apenas chaves e servem de índice, e as *externas* (folhas) guardam os registros. As folhas são encadeadas (`prox` e `ante`), o que permite percorrê-las em ordem.

```
typedef enum {Interna, Externa} TipoIntExt;
typedef struct TipoPagina {
    TipoIntExt Pt;
    union {
        struct { int ni; int ri[MM];
                 TipoApontador pi[MM + 1]; } U0;     // pagina interna
        struct { int ne; Registros re[MM2];
                 TipoApontador prox, ante; } U1;     // folha
    } UU;
} TipoPagina;
```

Código 10: Página da árvore B* (`arvoreBestrelaplus.h`)

Pesquisa. Nas páginas internas, a pesquisa usa "$\geq$" para escolher o ramo, pois a chave que sobe para o índice é uma cópia da primeira chave da folha da direita. Assim, toda pesquisa termina obrigatoriamente em uma folha, onde o registro é procurado.

```
if (Pag->Pt == Interna){
    i = 0;
    while (i < Pag->UU.U0.ni && Ch >= Pag->UU.U0.ri[i]) i++;
    return PesquisaEstrela(Ch, x, &Pag->UU.U0.pi[i], a);
}
i = 0;                                    // chegou a uma folha
while (i < Pag->UU.U1.ne && Ch > Pag->UU.U1.re[i].chave) i++;
if (i < Pag->UU.U1.ne && Ch == Pag->UU.U1.re[i].chave){
    *x = Pag->UU.U1.re[i];
    return true;
}
return false;
```

Código 11: Pesquisa na árvore B* (`arvoreBestrelaplus.h`), resumido

Inserção. Funciona como na árvore B, com uma diferença na divisão das folhas: em vez de o item do meio sair da folha, ele *permanece* na nova folha da direita e apenas uma cópia da sua chave sobe para a página interna. A divisão de páginas internas é igual à da árvore B.

```
Ap->UU.U1.ne = M2;                          // folha esquerda: M2 registros
for (i = 0, j = M2; i < M2; i++, j++){
    Ap->UU.U1.re[i]     = temp[i];
    ApTemp->UU.U1.re[i] = temp[j];
}
ApTemp->UU.U1.re[M2] = temp[MM2];
ApTemp->UU.U1.ne = M2 + 1;                  // folha direita: M2+1 registros
ApTemp->UU.U1.prox = Ap->UU.U1.prox;        // encadeia as folhas
ApTemp->UU.U1.ante = Ap;
Ap->UU.U1.prox = ApTemp;
RegRetorno->chave = ApTemp->UU.U1.re[0].chave;   // so a chave sobe
```

Código 12: Divisão de folha na árvore B* (`arvoreBestrelaplus.h`), resumido

2.8 Contagem dos quesitos

Tabela 2: O que é contado em cada método

Método	**Transferência**	**Comparação**
Acesso sequencial	Leitura de uma página de 100 registros	Chave com a tabela de índices e com cada registro da página
Árvore binária	Leitura ou escrita de um nó no arquivo da árvore	Chave procurada com a chave do nó
Árvore B	Leitura de um registro do arquivo de dados ou de uma página de `arvoreB.bin`	Chave com as chaves de uma página
Árvore B*	Leitura de um registro do arquivo de dados (a árvore fica em memória principal)	Chave com as chaves de uma página interna ou folha

3 Resultado de cada método individualmente:

Foi utilizado como métrica um tempo total de 60s por chave na pesquisa para timeout. Exceto para a pesquisa sequencial indexada porque a partir de vinte mil as pesquisa demoram muito

3.1 Acesso sequencial indexado:

3.1.1 Arquivo ordenado ascendentemente:

Tabela 3: Acesso sequencial indexado – arquivo ordenado ascendentemente (média de 20 chaves)

	Pré-processamento			Pesquisa		
Registros	**Transf.**	**Comp.**	**Tempo (ms)**	**Transf.**	**Comp.**	**Tempo (μs)**
100	1	0	0.017	1	51.50	13.98
200	2	0	0.032	1	52.00	13.78
2 000	20	0	1.556	1	61.00	12.32
20 000	200	0	18.285	1	151.00	17.91
200 000	2 000	0	173.441	1	1 051.00	19.40
2 000 000	20 000	0	1 752.720	1	10 051.00	31.62

3.1.2 Arquivos ordenados descendentemente e aleatoriamente:

Não se aplicam a este método: o acesso sequencial indexado exige que o arquivo esteja ordenado de forma crescente pela chave, pois a tabela de índices guarda a primeira chave de cada página e a pesquisa depende dessa ordem para escolher a página correta.

3.1.3 Análise (Acesso sequencial indexado):

O pré-processamento cresce de forma linear com o tamanho do arquivo: são feitas exatamente $\lceil n/100 \rceil$ transferências (1, 2, 20, ..., 20 000), uma por página lida para montar a tabela de índices, e nenhuma comparação. Na pesquisa, o número de transferências é sempre 1, independentemente de o arquivo ter 100 ou 2 000 000 de registros, pois a tabela de índices indica diretamente qual página deve ser lida. Por isso o tempo de pesquisa praticamente não varia (entre 12 e 32 μs).

O número de comparações, por outro lado, cresce linearmente: ele é composto pelas comparações com a tabela de índices, que é percorrida sequencialmente (em média metade das páginas), somadas às comparações dentro da página lida (em média metade dos 100 registros). Isso explica os valores de 51,5 com 100 registros (uma única página) e 10 051 com 2 000 000 (cerca de 10 000 comparações na tabela e 50 na página). Uma pesquisa binária na tabela de índices reduziria esse custo para $O(\log n)$ sem alterar o número de transferências.

O tempo médio de pré-processamento com 2 000 000 de registros (1,75 s) foi elevado pela primeira execução, que levou 28,4 s por ler o arquivo de cerca de 10 GB ainda fora do cache do sistema operacional; nas demais 19 execuções esse tempo ficou em torno de 0,35 s.

3.2 Árvore binária de pesquisa adequada a memória externa:

3.2.1 Arquivo ordenado ascendentemente:

Tabela 4: Árvore binária externa – arquivo ordenado ascendentemente (média de 20 chaves)

	Pré-processamento			**Pesquisa**		
Registros	**Transf.**	**Comp.**	**Tempo (ms)**	**Transf.**	**Comp.**	**Tempo (μs)**
100	5 249	5 049	1.876	50.50	50.50	20.45
200	20 499	20 099	6.665	100.50	100.50	35.77
2 000	2 004 999	2 000 999	616.622	1 000.50	1 000.50	315.27
20 000	*timeout (> 60 s por chave)*					
200 000	*não executado (tamanho menor já excedeu o timeout)*					
2 000 000	*não executado (tamanho menor já excedeu o timeout)*					

3.2.2 Arquivo ordenado descendentemente:

Tabela 5: Árvore binária externa – arquivo ordenado descendentemente (média de 20 chaves)

	Pré-processamento			**Pesquisa**		
Registros	**Transf.**	**Comp.**	**Tempo (ms)**	**Transf.**	**Comp.**	**Tempo (μs)**
100	5 249	5 049	1.892	50.50	50.50	21.82
200	20 499	20 099	6.749	100.50	100.50	35.04
2 000	2 004 999	2 000 999	627.058	1 000.50	1 000.50	336.28
20 000	*timeout (> 60 s por chave)*					
200 000	*não executado (tamanho menor já excedeu o timeout)*					
2 000 000	*não executado (tamanho menor já excedeu o timeout)*					

3.2.3 Arquivo ordenado aleatoriamente:

Tabela 6: Árvore binária externa – arquivo ordenado aleatoriamente (média de 20 chaves)

	Pré-processamento			**Pesquisa**		
Registros	**Transf.**	**Comp.**	**Tempo (ms)**	**Transf.**	**Comp.**	**Tempo (μs)**
100	991	791	0.742	8.10	8.10	7.41
200	2 247	1 847	1.584	8.95	8.95	9.05
2 000	31 642	27 642	20.379	12.95	12.95	11.27
20 000	408 220	368 220	257.770	18.30	18.30	20.01
200 000	5 002 129	4 602 129	3 444.851	22.45	22.45	28.09
2 000 000	*timeout (> 60 s por chave)*					

3.2.4 Análise (Árvore binária):

Nos arquivos ordenados (crescente e decrescente), os resultados são idênticos e confirmam o pior caso da árvore binária: cada nova chave é maior (ou menor) que todas as anteriores, a árvore degenera em uma lista e a pesquisa custa em média $(n+1)/2$ transferências, exatamente os valores 50,5, 100,5 e 1 000,5 obtidos. O pré-processamento cresce de forma quadrática, cerca de $n^2/2$ leituras: ao multiplicar o arquivo por 10 (de 200 para 2 000 registros), as transferências passaram de 20 499 para mais de 2 milhões, quase 100 vezes mais. Mantida essa proporção, 20 000 registros exigiriam cerca de 200 milhões de leituras de nós de 5 KB, o que explica o *timeout*.

No arquivo aleatório, a árvore fica razoavelmente balanceada e o custo da pesquisa cresce de forma logarítmica: de 8,1 transferências com 100 registros para 22,45 com 200 000. Esses valores ficam muito próximos do número médio teórico de comparações para uma árvore binária construída com chaves em ordem aleatória. Ainda assim, o pré-processamento é caro (mais de 5 milhões de transferências e 3,4 s com 200 000 registros), e com 2 000 000 de registros o tempo limite foi excedido.

3.3 Árvore B:

3.3.1 Arquivo ordenado ascendentemente:

Tabela 7: Árvore B ($m = 2$) – arquivo ordenado ascendentemente (média de 20 chaves)

	Pré-processamento			**Pesquisa**		
Registros	**Transf.**	**Comp.**	**Tempo (ms)**	**Transf.**	**Comp.**	**Tempo (μs)**
100	424	1 326	1.435	4.55	10.45	10.74
200	963	3 144	2.821	6.00	13.65	14.64
2 000	13 821	48 181	33.060	7.65	19.05	16.50
20 000	180 327	650 318	429.341	9.35	24.35	20.80
200 000	2 222 865	8 165 386	5 776.393	11.35	30.60	54.19
2 000 000	*timeout (> 60 s por chave)*					

3.3.2 Arquivo ordenado descendentemente:

Tabela 8: Árvore B ($m = 2$) – arquivo ordenado descendentemente (média de 20 chaves)

	Pré-processamento			**Pesquisa**		
Registros	**Transf.**	**Comp.**	**Tempo (ms)**	**Transf.**	**Comp.**	**Tempo (μs)**
100	424	1 342	1.542	4.55	10.20	11.92
200	963	3 066	3.083	6.00	13.55	15.48
2 000	13 821	43 493	39.230	7.65	18.90	18.65
20 000	180 327	561 055	458.756	9.35	24.05	23.89
200 000	2 222 865	6 879 919	6 288.015	11.35	30.30	76.39
2 000 000	*timeout (> 60 s por chave)*					

3.3.3 Arquivo ordenado aleatoriamente:

Tabela 9: Árvore B ($m = 2$) – arquivo ordenado aleatoriamente (média de 20 chaves)

	Pré-processamento			**Pesquisa**		
Registros	**Transf.**	**Comp.**	**Tempo (ms)**	**Transf.**	**Comp.**	**Tempo (μs)**
100	392	1 254	1.579	4.55	9.55	14.37
200	892	2 920	2.755	4.70	12.55	12.39
2 000	12 706	42 122	40.252	6.45	18.25	17.02
20 000	160 031	548 564	498.028	8.65	25.45	20.94
200 000	1 943 051	6 741 863	7 223.250	10.55	30.90	56.67
2 000 000	*timeout (> 60 s por chave)*					

3.3.4 Análise (Árvore B):

A pesquisa na árvore B apresenta crescimento logarítmico em todas as situações: o número de transferências passa de 4,55 com 100 registros para 10,55 a 11,35 com 200 000, enquanto o arquivo cresce 2 000 vezes. Esse valor corresponde às páginas lidas no caminho da raiz até a chave, mais uma leitura no arquivo de dados para obter o registro completo. As comparações seguem o mesmo comportamento (de cerca de 10 para cerca de 31), já que em cada página são comparadas no máximo $2m = 4$ chaves.

Ao contrário da árvore binária, a ordem dos dados quase não afeta o desempenho. Os arquivos crescente e decrescente produziram o mesmo número de transferências, ligeiramente maior que o do aleatório (11,35 contra 10,55 na pesquisa e 2,22 contra 1,94 milhões no pré-processamento). Isso ocorre porque, quando as chaves chegam em ordem, as páginas divididas ficam sempre com apenas m chaves e não voltam a receber inserções, gerando mais páginas e uma árvore um pouco mais alta.

O pré-processamento cresce na ordem de $n \log n$: cada inserção lê as páginas do caminho até a folha, o que dá cerca de 10 transferências por registro com 200 000 registros. Com 2 000 000 de registros, esse custo ultrapassou o tempo limite. O tempo de pesquisa, de 10 a 76 μs, é maior que o dos outros métodos porque cada página é lida do disco.

3.4 Árvore B*:

3.4.1 Arquivo ordenado ascendentemente:

Tabela 10: Árvore B* – arquivo ordenado ascendentemente (média de 20 chaves)

	Pré-processamento			Pesquisa		
Registros	**Transf.**	**Comp.**	**Tempo (ms)**	**Transf.**	**Comp.**	**Tempo (μs)**
100	100	1 104	2.301	0	9.15	0.69
200	200	2 579	2.376	0	9.85	5.48
2 000	2 000	38 545	28.112	0	13.25	2.27
20 000	20 000	511 087	316.036	0	17.75	4.17
200 000	200 000	6 383 339	3 306.246	0	21.00	3.64
2 000 000	*timeout (> 60 s por chave)*					

3.4.2 Arquivo ordenado descendentemente:

Tabela 11: Árvore B* – arquivo ordenado descendentemente (média de 20 chaves)

	Pré-processamento			Pesquisa		
Registros	**Transf.**	**Comp.**	**Tempo (ms)**	**Transf.**	**Comp.**	**Tempo (μs)**
100	100	657	1.039	0	8.30	0.75
200	200	1 442	2.600	0	9.80	1.40
2 000	2 000	18 793	19.681	0	13.15	2.24
20 000	20 000	230 289	254.156	0	14.95	3.39
200 000	200 000	2 722 817	2 374.320	0	19.35	3.94
2 000 000	*timeout (> 60 s por chave)*					

3.4.3 Arquivo ordenado aleatoriamente:

Tabela 12: Árvore B* – arquivo ordenado aleatoriamente (média de 20 chaves)

	Pré-processamento			**Pesquisa**		
Registros	**Transf.**	**Comp.**	**Tempo (ms)**	**Transf.**	**Comp.**	**Tempo (μs)**
100	100	896	6.393	0	9.00	0.72
200	200	2 095	8.003	0	9.70	0.84
2 000	2 000	28 245	33.951	0	13.35	1.23
20 000	20 000	359 671	304.310	0	17.30	2.01
200 000	200 000	4 366 440	3 125.327	0	21.50	2.45
2 000 000	*timeout (> 60 s por chave)*					

3.4.4 Análise (Árvore B*):

> Na árvore B*, o pré-processamento faz exatamente n transferências, uma por registro lido do arquivo de dados, e a pesquisa não faz nenhuma, pois toda a árvore é mantida em memória principal. Por esse motivo os tempos de pesquisa são os menores do trabalho, entre 0,7 e 5,5 μs; o valor de 5,48 μs no arquivo crescente com 200 registros se deve a uma única execução atípica (89 μs), e as demais ficaram em torno de 1 μs.

O número de comparações na pesquisa cresce de forma logarítmica (de 8 a 9 com 100 registros para 19 a 21,5 com 200 000) e é semelhante nas três situações, já que toda pesquisa desce obrigatoriamente até uma folha. Já no pré-processamento a ordem dos dados faz diferença: o arquivo crescente gerou 6,38 milhões de comparações, o aleatório 4,37 milhões e o decrescente 2,72 milhões. Isso acontece porque, nas páginas internas, as chaves são percorridas da esquerda para a direita: uma chave sempre maior que as anteriores percorre a página inteira em cada nível, enquanto uma chave sempre menor para na primeira comparação.

Com 2 000 000 de registros o método excedeu o tempo limite. Como as folhas armazenam os registros completos (cerca de 5 KB cada), a árvore ocuparia mais de 10 GB de memória principal, o que torna essa versão em memória inviável para arquivos desse tamanho.

4 Análise comparativa entre os métodos

Tabela 13: Comparação dos quatro métodos com 200 000 registros (média de 20 chaves)

		Pré-processamento		Pesquisa		
Método	**Arquivo**	**Transf.**	**Tempo (ms)**	**Transf.**	**Comp.**	**Tempo (μs)**
Acesso sequencial indexado	crescente	2 000	173.441	1	1 051.00	19.40
Árvore binária externa	aleatório	5 002 129	3 444.851	22.45	22.45	28.09
Árvore B ($m = 2$)	aleatório	1 943 051	7 223.250	10.55	30.90	56.67
Árvore B*	aleatório	200 000	3 125.327	0	21.50	2.45

Nota: o acesso sequencial indexado só se aplica a arquivos ordenados; para as árvores foi usado o arquivo aleatório, que é o único em que a árvore binária completou os 200 000 registros. Na árvore B*, a pesquisa não tem transferências porque a árvore é mantida em memória principal.

Acesso sequencial indexado. O acesso sequencial indexado destaca-se pela simplicidade e pela alta eficiência em arquivos ordenados: ao manter a tabela de páginas em memória principal, toda pesquisa custa exatamente uma transferência, qualquer que seja o tamanho do arquivo, e foi o único método que completou os 2 000 000 de registros. Por outro lado, como a tabela de índices é percorrida sequencialmente, o número de comparações cresce linearmente com o número de páginas (de 51,5 com 100 registros para 10 051 com 2 000 000). Sua principal limitação é a exigência de que o arquivo esteja ordenado, o que torna inserções e remoções custosas. Em relação aos demais métodos, supera com folga a árvore binária externa, mas perde em flexibilidade para as árvores B e B*, que se mantêm balanceadas mesmo com inserções em qualquer ordem.

Árvore binária (memória externa). Os testes evidenciam a falta de eficiência da árvore binária quando ela fica desbalanceada, o que ocorre quando é alimentada com arquivos ordenados: nesse caso cada inserção percorre toda a árvore, o custo de construção cresce de forma quadrática e o método não completou nem 20 000 registros dentro do tempo limite. Com o arquivo aleatório, a pesquisa é razoável (cerca de 22 transferências para 200 000 registros), mas a construção continua cara, pois cada nó lido corresponde a um registro inteiro de aproximadamente 5 KB. Há, portanto, uma grande discrepância entre o custo de montar a árvore e o de pesquisar uma chave: a pesquisa é rápida, mas paga-se um custo alto na construção inicial, e alternativas como o acesso sequencial indexado podem ser mais eficientes.

Árvore B. A árvore B mostrou o comportamento esperado de uma estrutura balanceada: o número de transferências na pesquisa cresce de forma logarítmica, passando de 4,55 com 100 registros para apenas 10,55 com 200 000 (arquivo aleatório), ou seja, um arquivo 2 000 vezes maior custou pouco mais que o dobro de acessos ao disco. Diferentemente da árvore binária, o desempenho praticamente não depende da ordem dos dados: com 200 000 registros, a pesquisa custou 11,35 transferências nos arquivos crescente e decrescente e 10,55 no aleatório. A pequena diferença vem do fato de que inserções em ordem deixam as páginas divididas com apenas m chaves, gerando mais páginas e uma árvore um pouco mais alta, o que também aparece no pré-processamento (2,22 milhões de transferências nos arquivos ordenados contra 1,94 milhão no aleatório). O ponto fraco é a construção: cada inserção lê do disco todas as páginas do caminho até a folha, e com 2 000 000 de registros o pré-processamento ultrapassou o tempo limite. Como a ordem utilizada é pequena ($m = 2$), a árvore fica mais alta do que ficaria com páginas maiores; uma ordem maior reduziria tanto a altura quanto o número de transferências.

Árvore B*. A árvore B* teve a pesquisa mais rápida entre todos os métodos, com cerca de 2 a 4 μs por chave com 200 000 registros, mais de dez vezes menos que a árvore B. Isso se deve principalmente ao fato de a árvore ser mantida em memória principal: o pré-processamento faz apenas as n leituras do arquivo de dados e a pesquisa não faz nenhuma transferência. Por isso, a comparação com a árvore B deve ser feita com cuidado, já que as duas não estão nas mesmas condições de acesso à memória. O número de comparações também cresce de forma logarítmica (de 9 com 100 registros para 21,5 com 200 000) e é menor que o da árvore B, embora toda pesquisa na B* precise descer até uma folha. A ordem dos dados influencia o pré-processamento: o arquivo crescente gerou 6,38 milhões de comparações e o decrescente apenas 2,72 milhões, porque a busca dentro das páginas internas é feita da esquerda para a direita e as chaves crescentes sempre percorrem a página inteira. A limitação deste método é o consumo de memória: como as folhas guardam os registros completos, com 2 000 000 de registros seriam necessários mais de 10 GB de memória principal, e esse caso não terminou dentro do tempo limite.

Árvore B $\times$ árvore B*. As duas estruturas garantem altura logarítmica independentemente da ordem de inserção, o que as torna muito superiores à árvore binária para arquivos ordenados. A árvore B mantém as páginas em disco e guarda apenas a chave e a posição do registro, o que a torna escalável para arquivos maiores que a memória, ao custo de mais transferências. A árvore B* separa índice e dados, concentrando os registros nas folhas encadeadas, o que favorece pesquisas e percursos em ordem, mas, na forma implementada, exige que todo o arquivo caiba em memória principal.

5 Conclusão

Os experimentos confirmaram, na prática, as complexidades estudadas para os quatro métodos. O acesso sequencial indexado foi o mais eficiente em transferências, com uma única leitura de disco por pesquisa em todos os tamanhos testados, inclusive 2 000 000 de registros, mas depende de o arquivo estar ordenado e tem um número de comparações que cresce linearmente com o tamanho da tabela de índices. A árvore binária de pesquisa em memória externa mostrou-se o método mais sensível à ordem dos dados: com arquivos aleatórios funciona de forma aceitável, mas com arquivos ordenados degenera em uma lista e não completou nem 20 000 registros dentro do tempo limite.

As árvores B e B* foram os métodos mais robustos, mantendo um custo de pesquisa logarítmico e praticamente independente da ordem dos dados. A árvore B, por manter suas páginas em disco, é a opção mais adequada quando o volume de dados excede a memória principal e há inserções frequentes; a árvore B* apresentou as pesquisas mais rápidas, porém, por manter a estrutura em memória, fica limitada pela quantidade de memória disponível.

De forma geral, conclui-se que não existe um método ideal para todos os casos: para arquivos estáticos e já ordenados, o acesso sequencial indexado é a escolha mais simples e eficiente; para arquivos dinâmicos ou sem ordenação prévia, as árvores B e B* são as mais indicadas, e a árvore binária só é recomendável quando se pode garantir que os dados chegam em ordem aleatória. Observou-se ainda que a primeira pesquisa de cada tamanho de arquivo levou mais tempo que as demais, por efeito do cache do sistema operacional, o que eleva as médias de tempo do pré-processamento; as contagens de transferências e de comparações, por outro lado, não são afetadas e permitem uma comparação mais confiável entre os métodos.

5.1 Impressões Gerais:

Por meio deste trabalho prático, foi possível desenvolver não apenas nossas habilidades relacionadas ao conteúdo de Estrutura de Dados, mas também nossas relações interpessoais com os colegas de equipe. Esse foi um aspecto de grande importância para todos os integrantes, uma vez que saber trabalhar em equipe é e sempre será um requisito fundamental no mercado de trabalho.

Durante o desenvolvimento do projeto, surgiram diversas dificuldades relacionadas à organização, visto que a equipe optou por utilizar uma ferramenta nova para a maioria de seus integrantes: o GitHub. Por meio dessa plataforma, conseguimos organizar o ambiente de desenvolvimento e distribuir as responsabilidades entre os membros da equipe. Entretanto, devido à inexperiência de grande parte dos integrantes com a ferramenta, ocorreram situações em que códigos foram sobrescritos ou desatualizados, resultando na utilização de algoritmos não funcionais e, consequentemente, no desperdício de tempo.

Além disso, a equipe precisou dedicar um tempo significativo à resolução de dificuldades técnicas, principalmente na construção da tabela de índices, no cálculo dos deslocamentos dentro do arquivo, na manipulação dos ponteiros dos nós armazenados em arquivo e na atualização dos nós filhos durante as inserções nas árvores. Também foram necessários estudos e ajustes para implementar a divisão de páginas quando estas atingiam sua capacidade máxima, a promoção de chaves para as páginas-pai, a redistribuição de chaves entre páginas irmãs e a realização de novas divisões sempre que necessário.